

Introducción a Hilos en Java

Programación Avanzada · UTPL · Abril–Agosto 2026

Asignatura	Programación Avanzada
Componente	Práctico Experimental — 2 horas
Repositorio	github.com/jorgaf/intro-threads (rama: version-a)
Tecnologías	Java 17 · Maven · IntelliJ IDEA / VS Code
Estudiante	
Fecha	

Objetivos del taller

Al finalizar este taller serás capaz de:

- Distinguir la ejecución secuencial de la concurrente observando el comportamiento real de `TimerTask`.
- Aplicar las tres alternativas de Java para crear hilos: `extends Thread`, `implements Runnable` y expresiones `lambda`.
- Decidir qué alternativa usar según el contexto del problema.
- Usar `join()` para coordinar la finalización de múltiples hilos.
- Documentar las diferencias observadas entre cada solución.

Parte 0 · Análisis del código base

El repositorio contiene dos clases. Léelas con atención antes de cualquier modificación.

TimerTask.java

```
// TimerTask.java – clase entregada
package ec.edu.utpl.carreras.computacion.proava;

public class TimerTask {
    private final String name;
    private int seconds;

    public TimerTask(String name, int seconds) {
```

```

        this.name = name;
        this.seconds = seconds;
    }

    public void run() {
        while (seconds > 0) {
            System.out.printf("%s: %d segundos restantes%n", name, seconds);
            seconds--;
        }
    }
}

```

MultiTimerConsole.java

```

// MultiTimerConsole.java – clase entregada
package ec.edu.utpl.carreras.computacion.proava;

public class MultiTimerConsole {
    public static void main(String[] args) {
        TimerTask timerA = new TimerTask("Temporizador A", 8);
        timerA.run();           // ejecución SECUENCIAL
        System.out.println();
    }
}

```

▶ Ejecuta la aplicación tal como está. Anota el resultado en la tabla de observaciones.

Análisis inicial — responde antes de continuar

a) **¿Cuántas líneas imprime la ejecución? ¿En qué orden aparecen?**

Imprimen 8 líneas, desde segundo mayor hasta llegar a 1.

b) **El método de TimerTask se llama run(). ¿Eso lo convierte ya en un hilo? Justifica**

No, que el método se llame run() no significa nada por sí solo. Para que algo sea un hilo en Java se necesita que la clase extienda Thread o implemente Runnable.

c) **Si agregas un segundo temporizador con timerA.run(); timerB.run(); ¿qué esperas que pase?**

Corren de forma secuencial, uno después del otro. Primero termina completamente el Temporizador A (sus 8 líneas), y solo cuando termina empieza el Temporizador B

Parte 1 · Alternativa A — extends Thread

¿Qué hace esta alternativa?

Limitación a recordar

TimerTask hereda de Thread. El JDK le da toda la maquinaria de un hilo. Al llamar start(), la JVM crea un hilo del SO y ejecuta run() en él.

Java NO permite herencia múltiple. Si TimerTask necesitara extender otra clase (ej. un widget de UI), esta alternativa NO sería viable.

Pasos

1. Crea la rama de trabajo:

```
// terminal
git checkout version-a
git checkout -b solucion-extends-thread
```

2. Abre TimerTask.java. Modifica la declaración de la clase en la línea 3:

```
// ANTES
public class TimerTask {

// DESPUÉS
public class TimerTask extends Thread {
```

3. Agrega super(name) como primera línea del constructor para asignar el nombre al hilo:

```
public TimerTask(String name, int seconds) {
    super(name);           // <-- línea nueva
    this.name = name;
    this.seconds = seconds;
}
```

4. El método run() ya existe. Solo agrega la anotación @Override para que el compilador verifique que sobrescribes correctamente:

```
// ANTES
public void run() {

// DESPUÉS
@Override
public void run() {
```

Tu clase completa debería verse así:

```
// TimerTask.java – versión A completa
// Cambio 1: la clase EXTIENDE Thread
```

```

public class TimerTask extends Thread {
    private final String name;
    private int seconds;

    public TimerTask(String name, int seconds) {
        super(name);          // Cambio 2: nombra el hilo con super()
        this.name = name;
        this.seconds = seconds;
    }

    // Cambio 3: se añade @Override — el método run() ES el hilo
    @Override
    public void run() {
        while (seconds > 0) {
            System.out.printf("%s: %d segundos restantes%n", name, seconds);
            seconds--;
        }
    }
}

```

5. Abre MultiTimerConsole.java. Reemplaza el contenido de main() para agregar tres temporizadores concurrentes:

```

// MultiTimerConsole.java — main() versión A
public static void main(String[] args) {
    TimerTask timerA = new TimerTask("Temporizador A", 8);
    TimerTask timerB = new TimerTask("Temporizador B", 5);
    TimerTask timerC = new TimerTask("Temporizador C", 3);

    timerA.start();    // start() crea el hilo y llama run() en él
    timerB.start();
    timerC.start();
}

```

 **NUNCA** llames run() directamente sobre el objeto. Llamar run() ejecuta el código en el hilo actual (¡comportamiento secuencial de siempre!). Solo start() crea un nuevo hilo del sistema operativo.

6. Ejecuta la aplicación. Observa la consola.

Observaciones — Parte 1 (extends Thread)

1. ¿Los mensajes de los tres temporizadores aparecen entremezclados? ¿Qué indica eso?

Si, eso indica que los tres hilos están corriendo **al mismo tiempo**, el sistema operativo les va dando turnos de CPU alternadamente.

2. ¿El orden de los mensajes es siempre el mismo entre ejecuciones? ¿Por qué?

No, el sistema operativo decide qué hilo corre en cada momento, y esa decisión puede variar en cada ejecución

3. ¿Qué pasaría si TimerTask necesitara también extender una clase llamada Widget?
Sería imposible, java no permite herencia múltiple, una clase solo puede extender una clase a la vez, si TimerTask extends Thread, ya no puede también extends Widget.

Parte 2 · Alternativa B — implements Runnable

¿Qué hace esta alternativa?

TimerTask implementa la interfaz Runnable. Eso significa que SABE qué hacer cuando se le pide que corra. Un objeto Thread externo es quien administra el hilo.

Ventaja clave

TimerTask puede extender cualquier otra clase Y además implementar Runnable. Separa la lógica de la tarea del mecanismo de ejecución.

Pasos

7. Crea la rama de trabajo (partiendo siempre de version-a):

```
// terminal
git checkout version-a
git checkout -b solucion-runnable
```

8. Modifica la declaración de TimerTask para que implemente Runnable:

```
// ANTES
public class TimerTask {

// DESPUÉS
public class TimerTask implements Runnable {
```

9. El constructor NO cambia. El método run() solo necesita @Override:

```
// TimerTask.java – versión B completa
// Cambio 1: la clase IMPLEMENTA Runnable
public class TimerTask implements Runnable {
    private final String name;
    private int seconds;

    public TimerTask(String name, int seconds) {
```

```

        this.name = name;        // constructor sin cambios
        this.seconds = seconds;
    }

    // Cambio 2: @Override – implementa el contrato de Runnable
    @Override
    public void run() {
        while (seconds > 0) {
            System.out.printf("%s: %d segundos restantes%n", name, seconds);
            seconds--;
        }
    }
}

```

10. Modifica main() en MultiTimerConsole.java. Ahora el Runnable se ENVUELVE dentro de un Thread:

```

// MultiTimerConsole.java – main() versión B
public static void main(String[] args) {
    TimerTask timerA = new TimerTask("Temporizador A", 8);
    TimerTask timerB = new TimerTask("Temporizador B", 5);
    TimerTask timerC = new TimerTask("Temporizador C", 3);

    // Cambio 3: el Runnable se ENVUELVE dentro de un Thread
    new Thread(timerA, "Hilo-A").start();
    new Thread(timerB, "Hilo-B").start();
    new Thread(timerC, "Hilo-C").start();
}

```



Nota la diferencia conceptual: en la Parte 1, timerA ES un Thread. En la Parte 2, timerA ES un Runnable que se le entrega a un Thread. La tarea y el mecanismo de ejecución están separados.

11. Ejecuta y verifica el comportamiento concurrente.

Observaciones — Parte 2 (implements Runnable)

1. ¿En qué línea exacta del código ocurre la diferencia respecto a la Parte 1 en el main()?

-timerA.start() porque timerA **es** un Thread.

-new Thread(timerA, "Hilo-A").start() porque timerA es un Runnable que se le **entrega** a un Thread.

2. ¿Por qué en esta versión escribimos new Thread(timerA, "Hilo-A").start() en lugar de timerA.start()?

Porque timerA ahora es un Runnable, **no un Thread**, la interfaz Runnable solo define el método run(), no tiene start(), para crear un hilo real necesitamos un objeto Thread, al que le pasamos el Runnable como argumento para decirle qué debe ejecutar.

3. Si TimerTask necesitara extender otra clase, ¿podría seguir usando Runnable? Explica.

Sí, esa es la gran ventaja de esta alternativa, en Java una clase puede extender solo una clase, pero puede implementar múltiples interfaces.

Parte 3 · Alternativa C — Expresiones lambda

¿Qué hace esta alternativa?

Runnable es una interfaz funcional (tiene un solo método abstracto). Desde Java 8 se puede reemplazar por una lambda, sin necesidad de modificar TimerTask.

Ventaja clave

TimerTask NO se toca. La lógica del hilo se define directamente en el lugar donde se usa. Ideal para tareas simples o cuando el código ya existe.

Anatomía de la expresión lambda

```
// Tres formas equivalentes de definir un Runnable
// Estas tres formas son EQUIVALENTES:

// 1. Clase anónima (forma antigua, Java 1.1)
new Thread(new Runnable() {
    public void run() { timerA.run(); }
}).start();

// 2. Lambda (Java 8+)
new Thread(() -> timerA.run()).start();

// 3. Referencia de método (Java 8+)
new Thread(timerA::run).start();
```

Pasos

12. Crea la rama de trabajo:

```
// terminal
git checkout version-a
git checkout -b solucion-lambda
```

13. TimerTask.java queda EXACTAMENTE igual que en el repositorio original. No hay absolutamente nada que cambiar en esa clase.

14. Modifica únicamente main() en MultiTimerConsole.java:

```
// MultiTimerConsole.java - main() versión C
public static void main(String[] args) {
    // TimerTask NO se modifica - se usa tal como está
    TimerTask timerA = new TimerTask("Temporizador A", 8);
    TimerTask timerB = new TimerTask("Temporizador B", 5);
    TimerTask timerC = new TimerTask("Temporizador C", 3);

    // Lambda: () -> expresión - implementa Runnable en línea
    new Thread(() -> timerA.run(), "Hilo-A").start();
    new Thread(() -> timerB.run(), "Hilo-B").start();
    new Thread(() -> timerC.run(), "Hilo-C").start();

    // Equivalente con referencia de método (forma más compacta):
    // new Thread(timerA::run, "Hilo-A").start();
}
```



La expresión `() -> timerA.run()` crea en línea un objeto que implementa `Runnable`. El compilador sabe que `Thread` espera un `Runnable`, y que `Runnable` solo tiene `run()`, por eso la lambda encaja perfectamente.

15. Ejecuta la aplicación. El comportamiento debe ser idéntico a las partes 1 y 2.

16. Como ejercicio adicional, reemplaza una de las lambdas por la sintaxis de referencia de método: `timerA::run`. Confirma que compila y funciona igual.



Observaciones — Parte 3 (Lambda)

1. ¿Qué cambios hiciste en `TimerTask.java`? (Se espera: ninguno.)

Ninguno, la clase se usa tal como fue entregada originalmente.

2. Escribe con tus propias palabras qué significa `() -> timerA.run()`

Es una **expresión lambda** que crea en línea un objeto que implementa `Runnable`.

- `()` → no recibe parámetros
- `->` → "haz esto:"
- `timerA.run()` → ejecuta el método `run()` de `timerA`

3. ¿Cuál de las tres alternativas consideras más legible? ¿Por qué?

La **lambda** es la más legible y concisa para tareas simples. Con una sola línea se crea y lanza el hilo sin modificar ninguna clase.

Parte 4 · Extensión — `join()` y `Thread.sleep()`

4A — Sincronizar con `join()`

Actualmente el hilo principal termina sin esperar a los temporizadores. `join()` permite al hilo principal pausarse hasta que otro hilo termine.

```
// main() con join()
Thread hiloA = new Thread(() -> timerA.run(), "Hilo-A");
Thread hiloB = new Thread(() -> timerB.run(), "Hilo-B");
Thread hiloC = new Thread(() -> timerC.run(), "Hilo-C");

hiloA.start();
hiloB.start();
hiloC.start();

try {
    hiloA.join(); // espera a que Hilo-A termine
    hiloB.join(); // espera a que Hilo-B termine
    hiloC.join(); // espera a que Hilo-C termine
} catch (InterruptedException e) {
    Thread.currentThread().interrupt();
}
System.out.println(">>> Todos los temporizadores han finalizado.");
```

i Si omites `join()`, el mensaje '>>> Todos los temporizadores han finalizado.' puede aparecer ANTES de que los hilos terminen, porque el hilo principal no espera a nadie.

4B — Contar en tiempo real con `Thread.sleep()`

Ahora la cuenta regresiva es instantánea. Modifica `run()` para que cuente un segundo real por iteración:

```
// TimerTask.java - run() con Thread.sleep()
// Modifica run() de TimerTask para que cuente en tiempo real
@Override
public void run() {
    while (seconds > 0) {
        System.out.printf("%s: %d segundos restantes%n", name, seconds);
        seconds--;
        try {
            Thread.sleep(1000); // pausa 1 segundo real
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            return;
        }
    }
    System.out.printf("%s: ¡Tiempo!%n", name);
}
```

Observaciones — Parte 4

1. Con `join()`, ¿el mensaje final siempre aparece al final? ¿Por qué?

Sí, siempre, porque `join()` obliga al hilo principal a **pausarse y esperar** a que cada hilo termine antes de continuar

2. Con `Thread.sleep(1000)`, ¿cuánto tarda en total la aplicación? ¿Coincide con el tiempo del temporizador más largo?

Tarda aproximadamente **8 segundos**, que es el tiempo del temporizador más largo (Temporizador A)

3. ¿Para qué sirve llamar `Thread.currentThread().interrupt()` dentro del catch de `InterruptedException`?

Cuando ocurre una `InterruptedException`, Java **limpia automáticamente la bandera de interrupción** del hilo, llamar `Thread.currentThread().interrupt()` la **restaura**, para que el código que esté más arriba en la pila de llamadas pueda saber que este hilo fue interrumpido y reaccionar apropiadamente.

Resumen comparativo de las tres alternativas

Criterio	extends Thread	implements Runnable	Lambda
¿Modifica <code>TimerTask</code> ?	Sí — hereda <code>Thread</code>	Sí — implementa <code>Runnable</code>	No
¿Puede extender otra clase?	No	Sí	Sí
¿Objeto <code>Thread</code> explícito?	No (es un <code>Thread</code>)	Sí — <code>new Thread(runnable)</code>	Sí — <code>new Thread(lambda)</code>
Requiere clase separada	Sí	Sí	No (inline)
Verbosidad	Media	Media	Mínima
Versión Java mínima	Java 1.0	Java 1.0	Java 8
Cuándo usarla	La clase ES un hilo por naturaleza	La clase TIENE una tarea concurrente y puede extender otro padre	Tareas simples, código ya existente, APIs modernas

Rúbrica de evaluación

Criterio	Excelente 10 pts	Satisfactorio 7 pts	En proceso 4 pts	No presentado 0 pts
----------	------------------	---------------------	------------------	---------------------

Parte 1 — extends Thread	Compila, ejecuta concurrente, usa start(), tiene @Override	Concurrente pero con detalles menores (sin super o @Override)	Compila pero sigue siendo secuencial	No entregado
Parte 2 — implements Runnable	Compila, Runnable envuelto correctamente en Thread, ejecuta concurrente	Funciona pero la envoltura Thread no está nombrada	Intento pero no concurrente	No entregado
Parte 3 — Lambda	Lambda correcta, TimerTask sin modificar, referencia de método probada	Lambda funciona, TimerTask no modificado, sin referencia de método	Lambda incorrecta o TimerTask modificado	No entregado
Observaciones (3 partes)	Todas respondidas con reflexión propia y precisa	Mayoría respondidas, algunas superficiales	Pocas respuestas o muy breves	Sin responder
Extensión: join() + sleep()	join() correcto, sleep() implementado, observaciones respondidas	Solo uno de los dos correctamente	Intento incompleto	No entregado

Instrucciones de entrega



Entrega tres ramas en tu fork del repositorio. Cada rama debe partir de version-a y representar una de las soluciones. El historial de commits debe mostrar el proceso, no solo el resultado final.

17. Haz fork del repositorio: <https://github.com/jorgaf/intro-threads>
18. Clona tu fork localmente: `git clone <URL-de-tu-fork>`
19. Asegúrate de comenzar desde la rama base: `git checkout version-a`
20. Para cada alternativa crea su rama, realiza los cambios, haz al menos un commit descriptivo y luego push.

```
// Flujo Git recomendado
# Ejemplo para la rama de la Parte 1
git checkout version-a
git checkout -b solucion-extends-thread
# ... realiza los cambios ...
git add .
git commit -m "Parte 1: TimerTask extiende Thread"
git push origin solucion-extends-thread
```

21. Repite el flujo para solucion-runnable y solucion-lambda.
22. Envía el enlace a tu repositorio de GitHub a través del espacio habilitado en Canvas.

¡Mucho éxito! Los hilos son uno de los pilares de las aplicaciones modernas. Con estos tres enfoques tienes las herramientas para enfrentar cualquier escenario de concurrencia básica. 🚀